



String Algorithms

Abdullah All Mamun Siam

Lecturer, Dhaka International University(DIU)

String Matching Introduction

String Matching Algorithms also known as String Searching Algorithms form an essential category of string algorithms. They are defined as methods used to identify the position where one or more specific strings occur within a larger string.

There are different types of method is used to finding the string

1. The Naive String Matching Algorithm
2. The Rabin-Karp-Algorithm
3. The Knuth-Morris-Pratt Algorithm

The Naive String Matching Algorithm

The Naive String Matching Algorithm is a straightforward and simple method for finding occurrences of a pattern within a larger text. It works by checking for a match between the pattern and every possible substring of the text, one character at a time.

Steps of the Algorithm:

1. Start at the beginning of the text.
2. Compare the pattern with a substring of the text, character by character.
3. If all characters in the pattern match the corresponding characters in the text, a match is found.
4. Move the pattern one position to the right in the text and repeat the comparison until the end of the text is reached.
5. Return the positions where matches are found.

Time Complexity

Best Case: $O(n)$, where the text and pattern do not match, and early termination occurs.

Worst Case: $O(n \cdot m)$, where n is the length of the text and m is the length of the pattern.

Example

Text: "ABABDABACDABABCABAB"

Pattern: "ABABCAB"

1. Align the pattern with the beginning of the text.
2. Compare each character of the pattern with the corresponding characters in the text.
3. Shift the pattern to the next position and repeat until the end of the text.

The Naive String Matching Algorithm is not the most efficient for long texts or complex patterns.

The Rabin-Karp Algorithm

The Rabin-Karp Algorithm is an efficient string matching algorithm that uses hashing to find a pattern within a text. Instead of checking substrings character by character, it calculates a hash value for the pattern and for substrings of the text, making comparisons faster.

The Rabin-Karp Algorithm Steps

1. Compute the hash value of the pattern using a specific hash function.
2. Compute the hash value for the first substring of the text with the same length as the pattern.
3. Compare the hash of the pattern with the hash of the current substring of the text. If the hashes match, perform a character-by-character comparison to confirm the match (to avoid false positives caused by hash collisions).
4. Slide the pattern over the text one position at a time, updating the hash and checking for matches.

Time Complexity

Best Case: $O(n+m)$, where n is the length of the text and m is the length of the pattern, assuming a good hash function.

Worst Case: $O(n \cdot m)$, in the case of many hash collisions.

Formula to calculate the hash value of a string

The hash value of a string in the **Rabin-Karp Algorithm** is typically calculated using a polynomial hash function. This function computes the hash value for a string by treating each character as a digit in a number system.

The formula is:

$$\text{Hash}(S) = (c_0 \cdot p^{m-1} + c_1 \cdot p^{m-2} + \dots + c_{m-1} \cdot p^0) \bmod q$$

Where:

- S is the string (e.g. the pattern or a substring of the text).
- $c_0, c_1, \dots, c_{m-1}$ are the numeric values of the characters in the string S.
For example, if the string contains ASCII characters, these are their ASCII values.
- p is a prime number or a base value used in the polynomial (e.g. 31 or 256).
- m is the length of the string S.
- q is a large prime number used to reduce hash collisions by taking the modulus.

Example of Hash Calculation

For the string $S="abc"$, using:

Base, $p=31$

Modulus, $q=101$

length of the string, $m=3$

ASCII values: $a=97$, $b=98$, $c=99$

$\text{Hash}("abc") = (97 \cdot 31^2 + 98 \cdot 31^1 + 99 \cdot 31^0) \bmod 101$